

Data Structures and Algorithms — Lecture Notes

Module 1: Algorithm Analysis

The running time of an algorithm is expressed as a function of the input size n . We ignore constant factors and lower order terms and describe the growth rate using asymptotic notation. Big-O gives an upper bound, Omega gives a lower bound and Theta gives a tight bound.

A loop that runs n times with constant work per iteration takes $O(n)$ time. Two nested loops that each run n times take $O(n^2)$. A loop in which the control variable is halved on every iteration runs $O(\log n)$ times.

Recurrences describe recursive algorithms. For example $T(n) = T(n/2) + O(1)$ solves to $O(\log n)$, while $T(n) = 2T(n/2) + O(n)$ solves to $O(n \log n)$.

Module 2: Searching and Sorting

Linear search examines the elements one by one until the key is found, so it takes $O(n)$ time in the worst case and works on unsorted data.

Binary search works only on a sorted array. It compares the key with the middle element. If they are equal the search ends. If the key is smaller, the search continues in the left half; otherwise it continues in the right half. Each comparison halves the search space, so after k comparisons at most $n / 2^k$ elements remain.

The recurrence for binary search is $T(n) = T(n/2) + O(1)$. The search stops when the remaining size reaches 1, that is when $n / 2^k = 1$, which gives $k = \log_2(n)$. Therefore the worst case time complexity of binary search is $O(\log n)$, the best case is $O(1)$ when the key is at the middle, and the auxiliary space is $O(1)$ for the iterative version.

Binary search cannot be applied to an unsorted array or to a plain linked list because random access to the middle element is required and the ordering of the data must be known. Sorting the data first costs $O(n \log n)$.

Merge sort divides the array into halves, sorts each half recursively and merges them; it takes $O(n \log n)$ time in every case and $O(n)$ extra space. Quick sort partitions around a pivot; its average time is $O(n \log n)$ but the worst case is $O(n^2)$ when the pivot is always the smallest or largest element.

Module 3: Stacks and Queues

A stack is a linear data structure in which insertion and deletion take place at one end only, called the top. It follows the LIFO principle: Last In, First Out. The main operations are push, pop and peek, and each takes $O(1)$ time.

Applications of stacks include the undo mechanism in text editors, the function call stack that supports recursion, evaluation of postfix expressions, conversion of infix to postfix, matching of balanced parentheses and browser back navigation.

A queue is a linear data structure in which insertion takes place at the rear and deletion at the front. It follows the FIFO principle: First In, First Out. The main operations are enqueue and dequeue, each $O(1)$ in a circular array or linked implementation.

Applications of queues include CPU and disk scheduling, printer spooling, buffering of data streams, and the breadth first traversal of graphs and trees.

Difference between a stack and a queue: a stack removes the most recently added element (LIFO) using one open end, whereas a queue removes the earliest added element (FIFO) using two ends. A pile of plates in a cafeteria is a stack; people standing in a line at a ticket counter form a queue.

Module 4: Linked Lists

A singly linked list is a sequence of nodes where each node stores data and a pointer to the next node. The list is accessed through a head pointer. Insertion at the head takes $O(1)$ time, while access by position and search take $O(n)$.

To reverse a singly linked list iteratively we keep three pointers: prev (initially null), curr (initially head) and next. In each step we save curr.next in next, set curr.next to prev, move prev to curr and curr to next. When curr becomes null, prev is the new head. The list is traversed once, so the time complexity is $O(n)$ and the extra space is $O(1)$.

A recursive reversal reverses the rest of the list and then makes the second node point back to the first. It also takes $O(n)$ time but uses $O(n)$ stack space for the recursion.

Module 5: Trees

A binary search tree keeps smaller keys in the left subtree and larger keys in the right subtree. Search, insertion and deletion take $O(h)$ time where h is the height, which is $O(\log n)$ for a balanced tree and $O(n)$ for a skewed tree. In-order traversal of a binary search tree visits the keys in sorted order.

Module 6: Graphs

A graph $G = (V, E)$ can be stored as an adjacency matrix, which needs $O(V^2)$ space, or as adjacency lists, which need $O(V + E)$ space.

Breadth first search (BFS) explores the graph level by level starting from a source vertex. It uses a queue. The algorithm marks the source visited and enqueues it; while the queue is not empty it dequeues a vertex, visits it and enqueues all its unvisited neighbours after marking them visited. The time complexity is $O(V + E)$ with adjacency lists. BFS finds the shortest path, in terms of number of edges, in an unweighted graph.

Depth first search (DFS) explores as far as possible along each branch before backtracking. It uses a stack, either explicitly or through recursion. Its time complexity is also $O(V + E)$. DFS is used for cycle detection, topological sorting and finding connected components, but it does not guarantee shortest paths.

Comparison of BFS and DFS: BFS uses a queue and DFS uses a stack; BFS visits vertices level by level while DFS goes deep first; BFS gives shortest paths in unweighted graphs while DFS does not; BFS needs more memory on wide graphs while DFS needs more on deep graphs. Both run in $O(V + E)$ time.

Module 7: Hashing

Hashing maps a key to an index of a table using a hash function, giving $O(1)$ average time for search, insertion and deletion. A good hash function distributes keys uniformly. A collision occurs when two different keys hash to the same index.

Separate chaining resolves collisions by keeping a linked list of all keys that hash to the same slot. Open addressing resolves collisions by probing for another free slot in the table itself; linear probing checks the next slots one by one, quadratic probing uses increasing squared offsets and double hashing uses a second hash function. The load factor is the ratio of stored keys to table size and controls performance.